

Résolution de Problèmes à base de Graphes d'États

Cours : Concepts d'Intelligence Artificielle

Etudiante : **Islem Souissi**

Niveau : L2 Informatique

Date : Mai 2026

Résumé du projet

Ce projet implémente la résolution de trois problèmes classiques d'IA (Loup-Chèvre-Salade, Seaux Die Hard, Missionnaires et Cannibales) à l'aide du solveur Talos basé sur les graphes d'états. Il comprend la modélisation XML, la conversion DOT/XML, l'affichage des solutions et l'automatisation via un LLM (Claude Sonnet).

Partie 1 — Modélisation XML avec Talos

1.1 Principe

Le solveur Talos résout des problèmes modélisés sous forme de graphes d'états. Chaque état est décrit par un ensemble de variables, et les transitions entre états sont définies dans une table de tuples (valmatrix). Le solveur cherche un chemin de l'état initial à l'état final en n étapes maximum.

1.2 Problème du Loup-Chèvre-Salade

Un fermier doit traverser une rivière avec un loup, une chèvre et de la salade. Le bateau ne peut transporter qu'un seul élément à la fois. Le loup mange la chèvre et la chèvre mange la salade si laissés seuls. Les variables sont L, C, S, B (0=gauche, 1=droite). Talos trouve 2 solutions en 9 étapes.

1.3 Problème des Seaux (Die Hard)

Avec un seau de 5L et un seau de 3L, obtenir exactement 4L. Les variables sont j5 (0..5) et j3 (0..3). Talos trouve 12 solutions en 10 étapes.

1.4 Problème des Missionnaires et Cannibales

5 missionnaires et 5 cannibales doivent traverser une rivière avec un bateau de capacité 3. Les cannibales ne doivent jamais être majoritaires sur une rive ni dans le bateau. Les variables sont M (0..5), C (0..5), B (0..1). Talos trouve 2 solutions en 10 étapes.

Problème	Variables	Solutions	n optimal
Loup-Chèvre-Salade	L,C,S,B ∈ {0,1}	2	9
Seaux Die Hard	j5∈{0..5}, j3∈{0..3}	12	10
Missionnaires	M,C∈{0..5}, B∈{0,1}	2	10

Partie 2 — Conversion DOT ↔ XML

2.1 DOT vers XML

Le programme `Dot_2_xml.py` lit un fichier DOT (graphe de transitions) et génère le fichier XML correspondant pour Talos. Il extrait les états (nœuds), les transitions (arêtes), l'état initial (`color=green`) et l'état final (`color=blue`). Chaque état reçoit un identifiant numérique unique.

Regex clé utilisé :

```
"([^\s"]*)" \s* -> \s* "([^\s"]*)"
```

2.2 XML vers DOT

La conversion inverse se fait par le fichier `reverse_convert.py`. Il lit la table de transitions XML et génère un graphe DOT avec les arêtes correspondantes, l'état initial en vert et l'état final en bleu.

2.3 Problème résolu : regex et structure XML

Le principal défi était le regex : l'ancien pattern `""?(.*?)""?` capturait mal les labels contenant le caractère `|` (ex: `"(LCSB|)"`). La correction utilise `""([^\s"]+)""` avec guillemets obligatoires. De plus, dans Talos chaque vararray doit être placé immédiatement après la déclaration de sa variable.

Partie 3 — Affichage des Solutions

Le programme `auto_pipeline.py` automatise la résolution et l'affichage. Il lance Talos avec l'option `-print 1` pour obtenir les états de la solution, extrait la première solution par regex, puis génère un fichier DOT où le chemin solution est mis en évidence en rouge gras. Le nœud initial est coloré en vert et le nœud final en bleu.

Extraction de la solution :

```
states = re.findall(r'\([^)]*\)', line) # ligne commençant par '('
```

Format DOT généré :

```
step0 [label="Etape 0\n(LCSB|)", color=green, fillcolor=lightgreen]; step0 -> step1  
[color=red, penwidth=3];
```

Résultats obtenus :

Problème	Solution affichée	Nœuds
Loup-Chèvre-Salade	(LCSB)→(LS CB)→...→(LCSB)	9
Seaux	(j3j5)→(jj5j3)→...	10
Missionnaires	(B MC)→...→(B MC)	10

Partie 4 — LLM et Prompts

4.1 LLM utilisé

Le LLM utilisé est Claude Sonnet (Anthropic). Ce modèle a été choisi pour sa capacité à générer des tables de transitions structurées à partir d'une description en langage naturel.

4.2 Prompt générique

Un prompt générique a été conçu pour s'adapter à tout problème de traversée ou de planification. Il spécifie les variables, le format de sortie (lignes de chiffres uniquement), et donne un exemple. Ce prompt est stocké dans prompt.txt et complété automatiquement par la description du problème.

```
Generate transitions for a state graph problem. Rules: states must be valid, forbidden states excluded, transitions legal. Output ONLY lines of numbers: val1 val2 ... valN val1' val2' ... valN' No text, no arrows, no explanations.
```

4.3 Pipeline d'automatisation

Le programme generate_problem.py automatise tout le processus :

Etape	Action	Outil
1	Description du problème (texte libre)	Utilisateur
2	Génération du prompt complet	prompt.txt
3	Appel au LLM → table de transitions	Claude.ai
4	Parsing des transitions (regex chiffres)	Python
5	Génération du fichier XML Talos	generate_problem.py
6	Résolution par Talos	Talos JVM
7	Extraction et affichage de la solution	auto_pipeline.py
8	Export DOT + PNG/JPG	Graphviz

4.4 Exemple — Loup-Chèvre-Salade via LLM

Le prompt a été soumis à Claude.ai avec la description du problème. Le LLM a généré 24 transitions valides. Après parsing et génération XML, Talos a trouvé 1 solution en 7 étapes : (LCS|B) → (LSB|C) → (LB|CS) → (LSB|C) → (LS|CB) → (LS|CB) → (|LCSB).

Conclusion

Ce projet a permis de mettre en oeuvre une chaîne complète de résolution de problèmes d'IA : modélisation par graphes d'états, résolution par solveur à contraintes (Talos), conversion entre formats (DOT/XML), visualisation des solutions et automatisation via un grand modèle de langage. Les trois problèmes (Loup-Chèvre-Salade, Seaux Die Hard, Missionnaires et Cannibales) ont été résolus avec succès. Le pipeline LLM démontre qu'un modèle de langage peut générer des tables de transitions correctes à partir d'une description en langage naturel, ouvrant la voie à une résolution entièrement automatisée de problèmes de planification.

Merci Pour Votre Attention